

Lab4 进程管理

一、练习1：分配并初始化一个进程控制块

1.struct proc_struct初始化

首先使用内核内存分配 `kmalloc(sizeof(struct proc_struct))` 分配 `proc_struct`

然后填充默认值，主要字段初始化为：

代码块

```
1      proc->state=PROC_UNINIT;
2      proc->pid=-1;
3      proc->runs=0;
4      proc->kstack=0;
5      proc->need_resched=0;
6      proc->parent=NULL;
7      proc->mm=NULL;
8      memset(&(proc->context),0,sizeof(struct context));
9      proc->tf=NULL;
10     proc->pgdir=boot_pgdir_pa;
11     proc->flags=0;
12     memset(proc->name,0,PROC_NAME_LEN+1);
```

- `proc->state=PROC_UNINIT`：未初始化态；
- `proc->pid=-1`：尚未分配PID；
- `proc->runs=0`：表示将该进程的调度次数计数器重置为0；
- `proc->kstack=0`：表示该进程的内核栈尚未被分配内存空间；
- `proc->need_resched=0`：在初始化时设为0，表示当前进程无需被调度，可继续执行；
- `proc->parent=NULL`：表示当前进程暂未关联父进程；
- `proc->mm=NULL`：表示不拥有独立地址空间，共享内核地址空间；
- `memset(&(proc->context),0,sizeof(struct context))`：通过memset将其初始化为全0，确保进程创建初期的上下文状态可预测、无残留数据，避免无效寄存器值导致的切换错误；
- `proc->tf=NULL`：尚未在内核栈上放置trapframe

- `proc->pgdir=boot_pgdir_pa`: 由于该内核线程运行于内核态, 因此直接复用uCore内核已建立的页表, 将其页目录指针设置为内核页表的起始地址`boot_pgdir`, 以此通过内核初始化阶段的地址转换有效性检查。

2.问题说明

(1) 请说明`proc_struct`中`struct context`成员变量含义和在本实验中的作用是什么?

成员变量含义: 内核级的被调度上下文, 保存了一组必要的`callee-saved`寄存器, 如返回地址`ra`、栈指针`sp`、以及`s0...s11`等寄存器, 用于内核态的上下文切换。

在本实验中的作用:

- 当内核在进程间切换时, 调度代码使用`switch_to(struct context *from, struct context *to)`来保存当前进程的寄存器到`from`, 并从`to`恢复寄存器, 从而实现上下文的切换。
- 在新建内核线程/进程时, 会设置新进程的`context.ra=forkret`和`context.sp=proc->tf`, 也就是把上下文栈指向内核栈上的`trapframe`。因此第一次被调度至该进程时, CPU会从`context.ra`指向的位置开始执行, 完成首次进入流程。
- 简言之, `context`是内核态的“程序计数器+保存寄存器集合”, 用于在内核中恢复执行流。

我们通过对代码进行调试来判断这一作用:

将代码 `memset(&(proc->context),0,sizeof(struct context));` 注释之后, 重新进行编译之后得到的结果如下图所示:

```
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
Base: 0x0000000000000000
Size: 0x0000000000000000 (128 MB)
End: 0x0000000000000000
DTB init completed
(THU.CST) os is loading ...

Special kernel symbols:
entry 0xc020004a (virtual)
etext 0xc0203e58 (virtual)
edata 0xc0209030 (virtual)
end 0xc020d4f0 (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
memory: 0x00000000, [0x00000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmallocc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en., Bye, Bye. :)"
kernel panic at kern/process/proc.c:395:
process exit!!.
```

```
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
Base: 0x0000000000000000
Size: 0x0000000000000000 (128 MB)
End: 0x0000000000000000
DTB init completed
(THU.CST) os is loading ...

Special kernel symbols:
entry 0xc020004a (virtual)
etext 0xc0203e66 (virtual)
edata 0xc0209030 (virtual)
end 0xc020d4f0 (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
memory: 0x00000000, [0x00000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmallocc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en., Bye, Bye. :)"
kernel panic at kern/process/proc.c:395:
process exit!!.
```

左图为注释后的结果, 右图为注释前的结果。`etext`为代码段的结束地址, 根据图中的结果可以说明, 当注释掉 `memset(&(proc->context),0,sizeof(struct context));` 这行代码后, `etext` 的值变小了, 其他的三个地址不变。其原因为 `memset(&(proc->context),0,sizeof(struct context));` 这行代码本身被编译进了`.text`段, 在进行编译的时候, 编译器为这一行代码生成若干条 `sd zero,xx(t1)` 指令, 链接器将这些指令放进`.text`段, 当我

们将这行代码注释之后，对应的机器指令从内核镜像中消失了，也就是说.text段变短了，因此etext地址发生了前移。

(2) 请说明proc_struct中struct trapframe *tf成员变量含义和在本实验中的作用是什么？

含义：保存一次陷入/中断/系统调用时的完整寄存器快照（包括通用寄存器、程序计数器EPC/PC、状态寄存器等）。通常放在进程的内核栈顶，用来中断返回时恢复用户态或内核态上下文。

代码块

```
1  struct trapframe
2  {
3      struct pushregs gpr;
4      uintptr_t status;
5      uintptr_t epc;
6      uintptr_t badvaddr;
7      uintptr_t cause;
8  };
```

以下是对trapframe的代码的具体的分析：

- struct pushregs gpr：通用寄存器的集合，保存了进程在中断发生时所有临时计算结果和数据指针，确保数据操作的连贯性，用来记录数据状态。
- uintptr_t status：为异常程序计数器，保存了导致异常或中断发生时正在执行的指令地址。它是进程恢复执行的唯一入口点。
- uintptr_t epc：为处理器状态寄存器，用来记录权限/模式，包含CPU的模式信息，是恢复进程执行环境的关键。
- uintptr_t badvaddr：为错误虚拟地址，记录了导致地址异常的虚拟地址，方便内核进行故障处理。

根据以上的代码的分析，我们可以确定其**在本实验中的作用**：

- tf用于保存中断或系统调用发生时的CPU状态，便于中断处理后正确恢复。
- 在创建新进程/线程时，会把一个临时构造好的内容复制到新进程的proc->tf，并在其中设置初始寄存器。因此新进程在第一次运行时可以依据tf的内容进入预期的入口。
- copy_thread将proc->tf放在内核栈顶，并把proc->context.sp指向该tf，这样forkret/forkrets能使用tf完成后续寄存器设置与返回。

二、练习2：为新创建的内核线程分配资源

1. 设计实现过程

为新创建的内核线程分配资源有以下步骤：

- 分配并初始化进程控制块

call `alloc_proc` to allocate a `proc_struct`

```
1  proc = alloc_proc();
2  if (proc == NULL) {
3      goto fork_out;
4  }
```

- 分配并初始化一个 `struct proc_struct`。
- 若失败，直接返回。

- 分配并初始化内核栈

call `setup_kstack` to allocate a kernel stack for child process

```
1  if (setup_kstack(proc) < 0) {
2      goto bad_fork_cleanup_proc;
3  }
```

- 为进程分配内核栈页面（`KSTACKPAGE`）。
- 若失败，跳到清理分支释放 `proc_struct`。

- 根据 `clone_flags` 决定是复制还是共享内存管理系统（`copy_mm` 函数）

call `copy_mm` to dup OR share mm according clone_flag

```
1  if (copy_mm(clone_flags, proc) < 0) {
2      goto bad_fork_cleanup_kstack;
3  }
```

- 复制或共享进程的内存。（在当前项目里此函数是空实现）
- 若失败，跳到清理分支释放已分配的内核栈和 `proc_struct`。但实际上本次实验应该不需要这个条件分支，因为Lab4的 `copy_mm()` 函数什么也不做，只会返回0。

- 设置进程的中断帧和上下文

call `copy_thread` to setup tf & context in `proc_struct`

```
1 copy_thread(proc, stack, tf);
```

- 在 `proc->kstack` 顶部建立trapframe (`proc->tf`)，把传入 `tf` 内容复制过去；
- `copy_thread` 这个函数设置child的返回值寄存器a0为0（子进程fork返回值为0）；
- 设定 `proc->tf->gpr.sp`（用户/线程栈指针）和 `proc->context`（`context.ra` 指向 `forkret`，`context.sp` 指向 `proc->tf`），保证进程第一次被调度时能从 `forkret/forkrets`入口继续执行；
- 所以实际上需要fork的东西是**stack**和**trapframe**。

• 设置进程的父进程和PID

代码块

```
1 proc->parent = current;
2 proc->pid = get_pid();
```

- 记录父进程指针，建立父子关系；
- 分配一个进程标识符pid。

• 把设置好的进程加入链表与哈希表

insert proc_struct into hash_list && proc_list

```
1 list_add(&proc_list, &(proc->list_link));
2 hash_proc(proc);
```

- 把新 `proc` 插入全局进程链表 `proc_list`，成为进程集的一员；
- 把 `proc` 按pid插入pid哈希表，便于按pid查找（`find_proc`）。

• 将新建的进程设为就绪态

call wakeup_proc to make the new child process RUNNABLE

```
1 wakeup_proc(proc);
```

- 将 `proc` 状态设置为PROC_RUNNABLE（可运行），让调度器可以选择执行它。

• 将返回值设为线程id（PID）

set ret vaule using child proc's pid

```
1 nr_process++;
```

```
2   ret = proc->pid;
```

- 增加全局进程数统计。
- 返回子进程的 pid 给调用者。

2. 能否做到给每个新fork的线程一个唯一的id?

- 能做到。
- `get_pid()` 函数的实现方式是：
 - a. 维护一个全局静态变量 `last_pid`（上次分配的pid），每次分配PID时从上次分配的位置继续往后找（`++last_pid`）；
 - b. 在PID空间（1 ~ MAX_PID - 1）中循环查找一个当前没有被进程占用的PID；
 - c. 每次找PID时都遍历当前所有的进程链表 `proc_list`，跳过已经被占用的PID；
 - d. 永远不会返回重复的PID（除非PID空间被占满）。
- 优化机制：`next_safe` 表示下一个安全的PID上限。
 - 如果某些PID已分配且大于 `last_pid`，`next_safe` 就记录第一个大于 `last_pid` 的PID。
 - 这样在冲突时无需扫描所有PID，可以直接跳到安全范围。

三、练习3：编写 `proc_run` 函数

1. 设计实现过程

`proc_run` 函数用于将指定的进程切换到 CPU 上运行。其主要任务是完成进程上下文的切换，包括更新当前进程指针、切换页表以及切换 CPU 寄存器状态。

1.1 检查与屏蔽中断

在进行进程切换之前，首先需要确保当前进程不是目标进程，并且为了保证切换过程的原子性，需要屏蔽中断。

代码块

```
1   if (proc != current) {
2       bool intr_flag;
3       local_intr_save(intr_flag);
4       // ...
5   }
```

分析：

- `if (proc != current)`：如果目标进程 `proc` 已经是当前运行的进程 `current`，则无需切换，直接返回。
- `local_intr_save(intr_flag)`：该宏定义在 `sync.h` 中。它首先读取 `sstatus` 寄存器保存当前的中断使能状态到 `intr_flag` 变量中，然后将 `sstatus` 的 `SIE` 位清零，从而禁用中断。这是为了防止在上下文切换的关键阶段被中断打断，导致内核状态不一致。

1.2 准备进程指针

定义指针变量指向当前进程和目标进程，方便后续操作。

代码块

```
1 struct proc_struct *prev = current;
2 struct proc_struct *next = proc;
```

1.3 切换页表

进程切换涉及地址空间的切换。在 RISC-V 中，这通过修改 `satp` (Supervisor Address Translation and Protection) 寄存器来实现。

代码块

```
1 c
2 if (next->pgdir != prev->pgdir) {
3     // 使用 write_csr 直接设置 satp 寄存器，避免 lsatp 宏在 64 位下的兼容性问题
4     uintptr_t satp_val = ((uintptr_t)SATP_MODE_SV39 << 60) | (next->pgdir >>
5     12);
6     write_csr(satp, satp_val);
7 }
```

分析：

- `next->pgdir != prev->pgdir`：只有当两个进程的页目录表不同时才需要切换页表。对于内核线程，它们通常共享内核页表（`boot_pgdir`），此时可能不需要切换。但为了通用性，这里进行了检查。

因此，我们选择直接使用 `write_csr` 指令来设置 `satp` 寄存器,而且由于

- **`satp` 寄存器的构造：**

`SATP_MODE_SV39 << 60`：将 Sv39 模式标识（8）移动到 `satp` 寄存器的高 4 位（Mode 字段）。`next->pgdir >> 12`：将页目录表的物理地址转换为物理页号（PPN）。两者按位或，构造出正确的 satp 值。

1.4 切换上下文与恢复中断

这是进程切换的核心步骤，包括更新 `current` 指针和硬件上下文的切换。

```
C
1  current = next;
2  switch_to(&(prev->context), &(next->context));
3  local_intr_restore(intr_flag);
```

分析：

- `current = next`：注意，`current=next` **必须在 `switch_to` 之前更新**。因为 `switch_to` 之后，CPU 就开始执行 `next` 进程的代码了（或者说是 `next` 进程上次暂停的地方）。如果 `next` 进程后续依赖 `current` 变量 `current` 必须已经指向它。
- `switch_to`：这是一个汇编函数（在 `switch.S` 中），它保存 `prev` 进程的 callee-saved 寄存器（`ra`, `sp`, `s0` - `s11`）到 `prev->context`，并从 `next->context` 恢复这些寄存器。
- `local_intr_restore(intr_flag)`：当进程切换回来（即 `prev` 再次变成 `current` 并从 `switch_to` 返回）时，恢复之前保存的中断状态。

2. 问题回答：在本实验的执行过程中，创建且运行了几个内核线程？

答：在本实验中，总共创建并运行了 2 个内核线程。

2.1 idleproc (PID = 0)

这是系统启动后的第一个内核线程。

代码块

```
1 // kern/process/proc.c :: proc_init()
2 if ((idleproc = alloc_proc()) == NULL) {
3     panic("cannot alloc idleproc.\n");
4 }
5 idleproc->pid = 0;
6 idleproc->state = PROC_RUNNABLE;
7 idleproc->kstack = (uintptr_t)bootstack;
8 idleproc->need_resched = 1;
9 set_proc_name(idleproc, "idle");
```

分析:

- `idleproc` 代表空闲进程。
- 它没有通过 `do_fork` 创建，而是直接通过 `alloc_proc` 分配结构体并手动初始化。
- 它的内核栈直接指向 `bootstack`（启动时的栈）。
- 当系统没有其他就绪进程时，调度器会运行 `idleproc`，它在 `cpu_idle` 函数中循环检查 `need_resched`。

2.2 initproc (PID = 1)

这是系统的第二个内核线程，用于执行初始化工作。

代码块

```
1 // kern/process/proc.c :: proc_init()
2 int pid = kernel_thread(init_main, "Hello world!!", 0);
3 initproc = find_proc(pid);
4 set_proc_name(initproc, "init");
```

分析:

- `initproc` 是通过 `kernel_thread` 创建的。
- `kernel_thread` 内部调用了 `do_fork`，这会分配新的 `proc_struct` 和内核栈，并设置上下文。
- `initproc` 的入口函数是 `init_main`，它会打印 "Hello world!!"。

2.3 执行流程总结

1. **启动**: `kern_init` 调用 `proc_init`。
2. **创建**: `proc_init` 初始化 `idleproc`，然后通过 `kernel_thread` 创建 `initproc`。
3. **调度**: `kern_init` 最后调用 `cpu_idle`（此时运行的是 `idleproc`）。
4. **切换**: `cpu_idle` 发现 `need_resched` 为 1，调用 `schedule`。调度器选择 `initproc`，调用 `proc_run` 切换到 `initproc`。
5. **运行**: `initproc` 执行 `init_main`，输出字符串。
6. **退出**: `init_main` 结束，`do_exit` 被调用，`initproc` 变为 `PROC_ZOMBIE`，再次调用 `schedul` 切换回 `idleproc`。

四、扩展练习 Challenge

扩展练习 Challenge

1. 说明语句 `local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的？

1.1 `local_intr_save` 的实现

`local_intr_save` 宏用于保存当前中断状态并禁用中断。

代码块

```
1 // kern/sync/sync.h
2 static inline bool __intr_save(void) {
3     if (read_csr(sstatus) & SSTATUS_SIE) {
4         intr_disable();
5         return 1;
6     }
7     return 0;
8 }
```

```

9
10 #define local_intr_save(x) \
11     do {                      \
12         x = __intr_save(); \
13     } while (0)

```

分析：

- `read_csr(sstatus) & SSTATUS_SIE`：检查当前 `sstatus` 寄存器的 `SIE` (Supervisor Interrupt Enable) 位。如果为 1，表示当前中断是开启的。
- `intr_disable()`：如果中断开启，调用此函数关闭中断。该函数内部执行 `clear_csr(sstatus, SSTATUS_SIE)`，将 `SIE` 位清零。
- `return 1` / `return 0`：返回之前的状态（1 表示开启，0 表示关闭）。
- `x = __intr_save()`：将返回值保存到变量 `x`（即 `intr_flag`）中。这使得我们知道在进入临界区之前，中断到底是开着的还是关着的。

1.2 `local_intr_restore` 的实现

`local_intr_restore` 宏用于恢复之前保存的中断状态。

代码块

```

1 // kern/sync/sync.h
2 static inline void __intr_restore(bool flag) {
3     if (flag) {
4         intr_enable();
5     }
6 }
7
8 #define local_intr_restore(x) __intr_restore(x);

```

分析：

- `if (flag)`：检查之前保存的状态 `flag`。
- `intr_enable()`：如果 `flag` 为 1（说明之前中断是开启的），则调用此函数重新开启中断。该函数内部执行 `set_csr(sstatus, SSTATUS_SIE)`，将 `SIE` 位置 1。

* **保持关闭**：如果 `flag` 为 0（说明之前中断本来就是关闭的），则什么也不做，保持中断关闭状态。

1.3 机制总结

这种设计的主要目的是支持**嵌套的中断禁用**（Nested Interrupt Disabling）。

- 如果函数 A 禁用了中断，然后调用函数 B。
- 函数 B 也使用 `local_intr_save`。它会发现中断已经是关闭的，所以保存的 `intr_flag` 为 0。
- 函数 B 执行完 `local_intr_restore` 时，检查 `intr_flag` 为 0，所以**不会**开启中断。
- 回到函数 A，函数 A 继续执行，直到它自己调用 `local_intr_restore`，此时它的 `intr_flag` 为 1，才会真正重新开启中断。

这保证了临界区的安全性，防止被调用的子函数意外提前开启中断。

2. 深入理解不同分页模式的工作原理

2.1 `get_pte()` 中两段相似代码的分析

在 `kern/mm/pmm.c` 的 `get_pte` 函数中，存在两段逻辑非常相似的代码。

第一段代码（Level 2 页表处理）：

代码块

```
1  pde_t *pdep1 = &pgdir[PDX1(la)];
2  if (!(*pdep1 & PTE_V)) {
3      struct Page *page;
4      if (!create || (page = alloc_page()) == NULL) {
5          return NULL;
6      }
7      // ... 初始化页面 ...
8      *pdep1 = pte_create(page2ppn(page), PTE_U | PTE_V);
9  }
```

第二段代码（Level 1 页表处理）：

代码块

```
1  pde_t *pdep0 = &((pte_t *)KADDR(PDE_ADDR(*pdep1)))[PDX0(la)];
2  if (!(*pdep0 & PTE_V)) {
3      struct Page *page;
4      if (!create || (page = alloc_page()) == NULL) {
5          return NULL;
6      }
7      // ... 初始化页面 ...
8      *pdep0 = pte_create(page2ppn(page), PTE_U | PTE_V);
9  }
```

分析：

- **Sv39 分页机制：**RISC-V 的 Sv39 模式使用三级页表结构（Level 2 -> Level 1 -> Level 0）。
- **递归结构：**多级页表的每一级（除了最后一级）都指向下一级页表的物理基址。因此，查找或创建下一级页表的过程是重复的。第一段代码处理 Level 2 页表项（`pdep1`），如果不存在，分配一个物理页作为 Level 1 页表，并建立映射。第二段代码处理 Level 1 页表项（`pdep0`），如果不存在，分配一个物理页作为 Level 0 页表，并建立映射。

- **异同点：**

Sv32 (二级页表)：只需要一段这样的代码（Root -> Leaf）。

Sv39 (三级页表)：需要两段这样的代码（Root -> Mid -> Leaf）。

Sv48 (四级页表)：需要三段这样的代码。

代码逻辑完全一致，只是索引计算宏（`PDX1` vs `PDX0`）和指针层级不同。

2.2 `get_pte()` 功能合并的优缺点

目前 `get_pte` 将“查找 PTE”和“分配 PTE”合并在一个函数中，通过 `create` 参数控制。

优点：代码复用：页表遍历的前半部分逻辑（计算索引、检查存在位）对于查找和分配是完全一样的。合并可以减少代码重复。

接口简洁：上层调用者（如 `page_insert`）只需要调用一个函数就能完成“找到或创建”的操作，逻辑流更连贯。

缺点：

职责不单一：违反了单一职责原则（Single Responsibility Principle）。一个函数同时负责查询和修改（分配内存）。

总结：

在 uCore 这种教学操作系统中，为了保持代码的紧凑和易读性，这种合并写法是**可以接受且高效的**。但在工业级内核（如 Linux）中，通常会将页表遍历（Walk）和内存分配逻辑分层处理，以提供更细粒度的控制和更高的安全性。

五、实验中的重要知识点

1. 内核线程与用户进程

比较项	运行模式	地址空间
内核线程	仅在内核态运行	共享内核地址空间
用户进程	在用户态和内核态之间切换	拥有独立的用户虚拟地址空间

注意：ucore内核中的所有内核线程共享一个内核地址空间和其他资源，所以这些内核线程从属于同一个**唯一的内核进程**，即ucore内核本身。

2. 进程状态

2.1 uCore中的进程状态

- **PROC_UNINIT（未初始化）**：进程控制块刚被创建，尚未完成初始化。
- **PROC_RUNNABLE（就绪/运行）**：进程已准备好运行，可能正在等待CPU或正在CPU上执行。
- **PROC_SLEEPING（睡眠/阻塞）**：进程因等待某个事件或资源而暂时无法执行。
- **PROC_ZOMBIE（僵尸）**：进程已执行完毕，但其资源尚未被父进程回收。

在 `schedule` 函数中，只有处于 `PROC_RUNNABLE` 状态的进程才会被选中执行。

uCore之外，常见的“进程状态模型”主要有**三状态模型**和**五状态模型**。

2.2 三状态模型

1. Ready（就绪）：进程已准备好运行，只要CPU一来就能立即执行。
 - 已经创建好、已分配必要资源（内存、PCB等）
 - 但暂时没有CPU

- 2. Running（运行）：进程正在占用CPU，真正执行指令。
- 3. Blocked / Waiting（阻塞）进程暂时不能运行，它会被放到阻塞队列（等待队列）。典型原因：
 - 在等待 I/O（如磁盘、网络）
 - 在等待互斥锁、信号量
 - 在等待某个事件发生

2.3 五状态模型

- 1. New（新建）：进程刚被创建，系统正在分配它的资源。
- 2. Ready（就绪）：同三状态模型。
- 3. Running（运行）：同三状态模型。
- 4. Blocked / Waiting（阻塞）：同三状态模型。
- 5. Exit / Terminated（终止）：进程执行结束，或被kill，但系统还未完全回收它的资源。

3. Round-Robin轮转调度

Round-Robin（RR）轮转调度给每个进程一个固定长度的 CPU 时间片（time slice），用完就强行切走，轮着来。所有进程排成一个循环队列，挨个执行。

就绪队列：P1 → P2 → P3 → P4 → ... → Pn → (回到P1)

调度器：取队头进程（例如 P1），给它固定时间片，时间片用完 → 强制中断 → 放到队尾，换下一个（P2），周而复始……

优点：① 公平：所有进程平均分配 CPU，没有饥饿问题。

② 响应快：适用于多任务交互场景。

③ 实现简单：只需要循环队列和定时器中断。

缺点：① 不考虑优先级：所有进程都一样重要，无法对重要任务加速。（现代操作系统可优化，如 MLFQ）

② 频繁抢占会导致高切换开销：特别是时间片太短时。

③ 对长任务不友好：CPU密集型任务会被不断打断。

六、其他重要知识点

1. 进程与线程

项目	进程	线程
调度单位	可以被调度	最小调度单位

地址空间	互相独立	共享同一地址空间
内存	不共享（安全）	共享（高效但不安全）
创建成本	高（需要复制页表）	低（共享内存即可）
上下文切换成本	大	小
崩溃影响	不影响其他进程	会导致整个进程崩溃

2. 进程挂起

挂起是操作系统把一个进程暂时“冻结”，并且可能把它从内存**移到磁盘**，以释放内存。该进程不会获得 CPU，也不能继续运行，直到再次被恢复（resume）。

挂起类型：

- **就绪挂起（Ready Suspended）**

- 进程本来是就绪状态，但被OS挪到磁盘（换出）。
- 它本来可以执行，但内存不够 → 被挂起。

- **阻塞挂起（Blocked Suspended）**

进程本来是在等待某件事（比如IO），同时OS又把它从内存挪出去。

典型情景：

- 一个进程正在等磁盘IO
- 内存紧张
- OS发现“反正它也不能执行，不如把它换出”

3. 抖动问题（thrashing）

抖动问题是进程物理页面太少，不能包含工作集，造成**大量缺页**，花大量时间在换页上，而几乎没有时间真正运行进程。此时CPU利用率极低，系统基本卡死，但磁盘狂转。

产生原因：随着驻留内存的进程数目增加，分配给每个进程的物理页面数不断减小，导致**内存不够**，但此时系统里还运行着很多需要大量内存的进程。

操作系统需在并发水平和缺页率之间达到一个平衡！